College Dublin
Computing • IT • Business

Assessment Cover Page

Student Full Name

Valeria da Silva Dionisio

Student Number

2025627

Module Title

Software Development Fundamentals

Assessment Title

Integrated CA2 – 55%

Lecturer/Supervisor

Ken Healy

Assessment Due Date

23/05/2026

Date of Submission

22/05/2026

Use of AI Tools

I used ChatGPT mainly to help improve the grammar of my writing and organise my ideas throughout the report. I also used it to help me better understand some UML concepts and how to use Draw.io.

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on academic misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source.

I declare it to be my own work and that all material from third parties has been appropriately referenced.

I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other

Contents

Introduction 1

1. Use Cases and Use Case Diagrams 2

2. UML Model 1 – Class Diagram 7

3. UML Model 2 - Activity Diagram 8

4.UML Model 3 – Component Diagram 10

5. Requirements 10

5.1 Functional Requirements 10

5.2 Non-Functional Requirements 11

6. User stories 11

7. Unit Test 14

8. References 17

9. Appendix 18

Introduction

This report is about the planning and design of a Bank Organisation System prototype. The available organisation options were School, Bank or Department Store. I decided to choose the Bank option because it clearly connects with employees, managers, departments and hierarchy, which are important parts of the system.

The system will be a command-line program where the user can work with employee records, manager types and bank departments. Before starting the full Java implementation, the system is modelled using use cases, UML diagrams, requirements, user stories and unit test planning.

1. Use Cases and Use Case Diagrams

UC1 – Read Employee Records

Use Case ID	UC01	
Description (or Overview)	Read and load employee records from Applicants_Form.txt into the system.	
Dependencies	NONE	
Precondition(s)	The file must exist and contain valid employee records.	
Primary Actor	Bank Administrator	
Main Sequence << describes the actions that take place in a 'normal interaction>>	**Step**	**Action**
	1	The administrator launches the program
	2	The system asks the administrator to enter the filename.
	3	The administrator enters Applicants_Form.txt.
	4	The system reads the file.
	5	The system stores the employee records in memory.
	6	The system displays "File read successfully".
Postcondition(s)	Employee records are loaded and available for sorting, searching and hierarchy creation.	
Alternative Paths (or Exceptions) <<sets out alternative sequence(s) of actions in unusual situations, such as a user error If there are none then this section would be omitted >>	**Step 3**	**Action**
	a	The administrator enters an incorrect filename
	b	The system displays an error message.
	c	The system asks the administrator to try again.
	Step 4	**Action**
	a	The file is empty or contains invalid data.
	b	The system displays a warning message.
Comments (or Notes)	This use case is needed because the system needs employee data before the other functions can work	

UC2 – Sort Employee Records

Use Case ID	UC02	
Description (or Overview)	Sort the loaded employee records alphabetically and display the first 20 results.	
Dependencies	UC01 – Read Employee Records	
Precondition(s)	Employee records must already be loaded into the system.	
Primary Actor	Bank Administrator	
Main Sequence << describes the actions that take place in a 'normal interaction>>	**Step**	**Action**
	1	The administrator selects the SORT option from the menu.

	2	The system confirms that SORT was selected.
	3	The system sorts the employee records alphabetically.
	4	The system displays the first 20 sorted employee names.
	5	The system returns to the main menu.
Postcondition(s)	The employee records are sorted alphabetically and ready for searching.	
Alternative Paths (or Exceptions) <<sets out alternative sequence(s) of actions in unusual situations, such as a user error If there are none then this section would be omitted >>	**Step 1**	**Action**
	a	The administrator selects SORT before loading employee records.
	b	The system displays an error message.
	c	The system returns to the main menu.
	Step 4	**Action**
	a	The list contains less than 20 employee records.
	b	The system displays all available records instead.
Comments (or Notes)	This use case is important because sorting makes the employee list easier to read and prepares it for searching.	

UC3 – Search Employee by Name

Use Case ID	UC3	
Description (or Overview)	Search for an employee by name and display their information.	
Dependencies	UC01 – Read Employee Records UC02 – Sort Employee Records	
Precondition(s)	Employee records must already be loaded and sorted.	
Primary Actor	Bank Administrator	
Main Sequence << describes the actions that take place in a 'normal interaction>>	**Step**	**Action**
	1	The administrator selects the SEARCH option from the menu.
	2	The system asks the administrator to enter an employee name.
	3	The administrator enters the employee name.
	4	The system searches for the employee record.
	5	The system displays the employee information, including manager type and department.
	6	The system returns to the main menu.
Postcondition(s)	The employee information is displayed successfully.	
Alternative Paths (or Exceptions)	**Step 3**	**Action**

<<sets out alternative sequence(s) of actions in unusual situations, such as a user error If there are none then this section would be omitted >>	a	The administrator enters an empty name.
	b	The system displays a validation message.
	c	The system asks the administrator to try again.
	Step 4	**Action**
	a	The employee record is not found.
	b	The system displays “Employee not found”.
Comments (or Notes)	This use case allows the user to quickly find employee information without checking the full employee list manually	

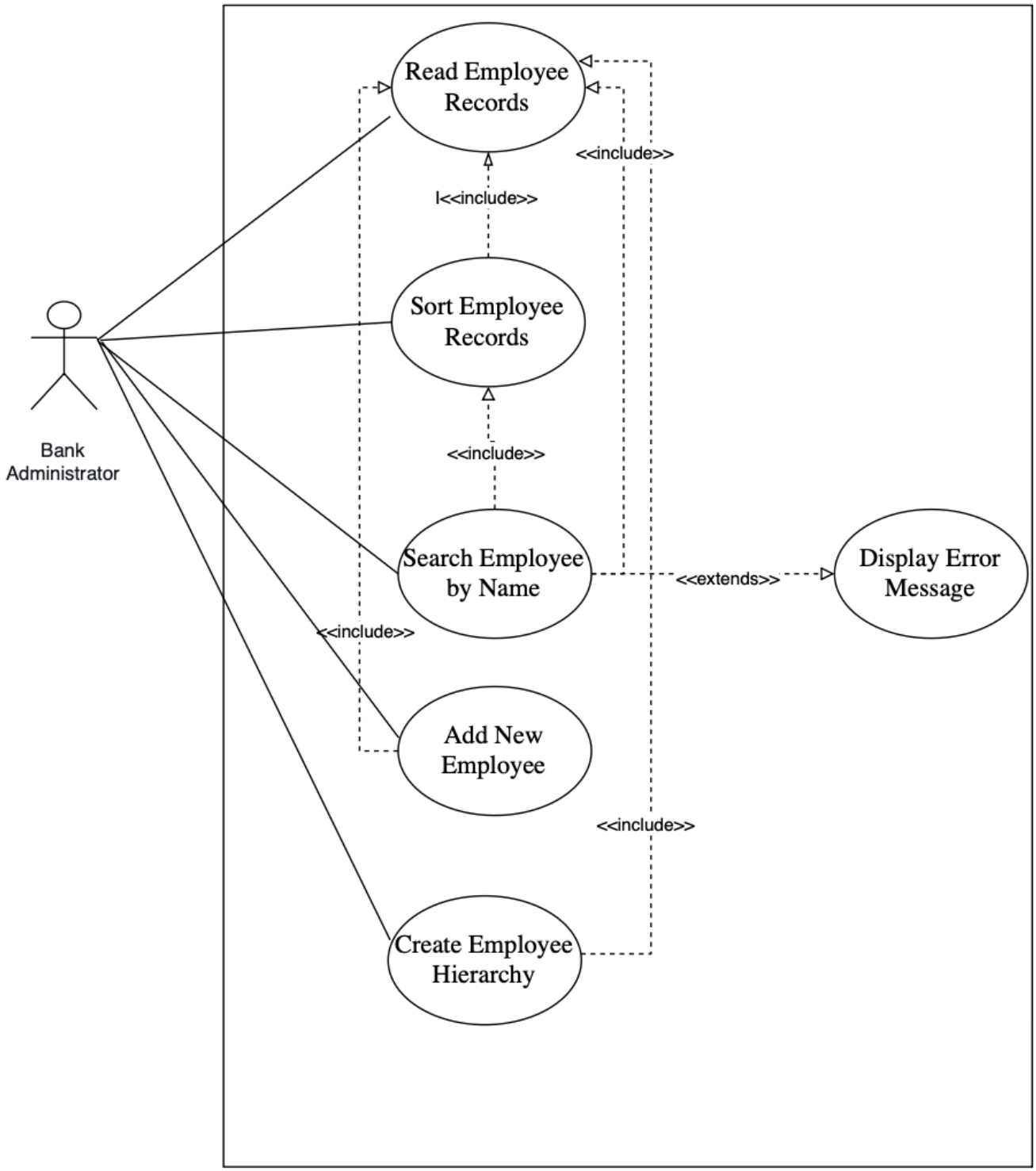
UC4 – Add New Employee

Use Case ID	UC4	
Description (or Overview)	Add a new employee record to the system	
Dependencies	UC01 – Read Employee Records	
Precondition(s)	Employee records should already be loaded into the system.	
Primary Actor	Bank Administrator	
Main Sequence << describes the actions that take place in a ‘normal interaction>>	**Step**	**Action**
	1	The administrator selects the ADD RECORD option from the menu.
	2	The system asks the administrator to enter the employee name.
	3	The administrator enters the employee name.
	4	The system asks the administrator to select a manager type and department.
	5	The administrator selects a valid manager type.
	6	The administrator selects a valid department
	7	The system adds the new employee record and displays a confirmation message.
Postcondition(s)	The new employee record is added to the system.	
Alternative Paths (or Exceptions) <<sets out alternative sequence(s) of actions in unusual situations, such as a user error If there are none then this section would be omitted >>	**Step 3**	**Action**
	a	The administrator enters an empty name.
	b	The system displays a validation message.
	c	The system asks the administrator to try again.
	Step 5	**Action**
	a	The administrator selects an invalid manager type or department.
	b	The system displays an error message.
Comments (or Notes)	This use case is needed so the employee list can be updated when new staff are added.	

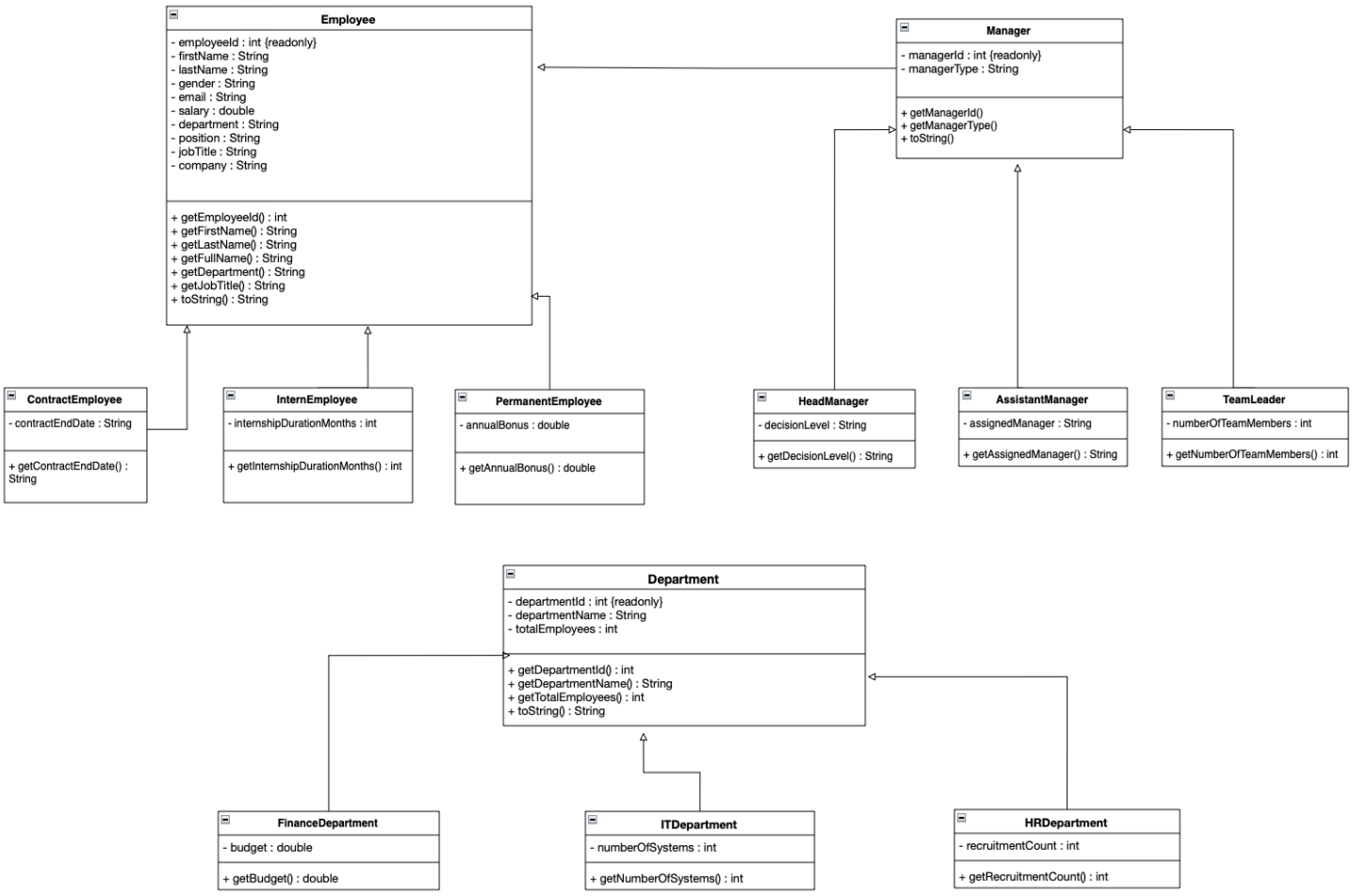
UC5 – Create Employee Hierarchy

Use Case ID	UC5	
Description (or Overview)	Create and display an employee hierarchy using a binary tree structure.	
Dependencies	UC01 – Read Employee Records	
Precondition(s)	The system must contain at least 20 employee records.	
Primary Actor	Bank Administrator	
Main Sequence << describes the actions that take place in a 'normal interaction>>	**Step**	**Action**
	1	The administrator selects the CREATE BINARY TREE option from the menu.
	2	The system retrieves the employee records.
	3	The system creates the binary tree hierarchy.
	4	The system displays the hierarchy, tree height and total node count.
	5	The system returns to the main menu
Postcondition(s)	The employee hierarchy is successfully created and displayed.	
Alternative Paths (or Exceptions) <<sets out alternative sequence(s) of actions in unusual situations, such as a user error If there are none then this section would be omitted >>	**Step** 2	**Action**
	a	The system contains fewer than 20 employee records.
	b	The system displays an error message.
	c	The system returns to the main menu.
	Step	**Action**
Comments (or Notes)	This use case is important because it represents the organisation hierarchy and demonstrates the use of binary tree structures in the project.	

I created The Use Case Diagram based on the example provided by the lecturer on Moodle, where <<include>> relationships were also used to represent dependencies between actions. During class, it was explained that multiple use cases could be grouped into one diagram instead of creating a separate diagram for each use case, helping to avoid repetition and making the system interactions easier to understand.


Read Employee Records
<<include>>
I<<include>>
Sort Employee Records
<<include>>
Bank Administrator
Search Employee by Name
<<extends>>
Display Error Message
<<include>>
Add New Employee
<<include>>
Create Employee Hierarchy

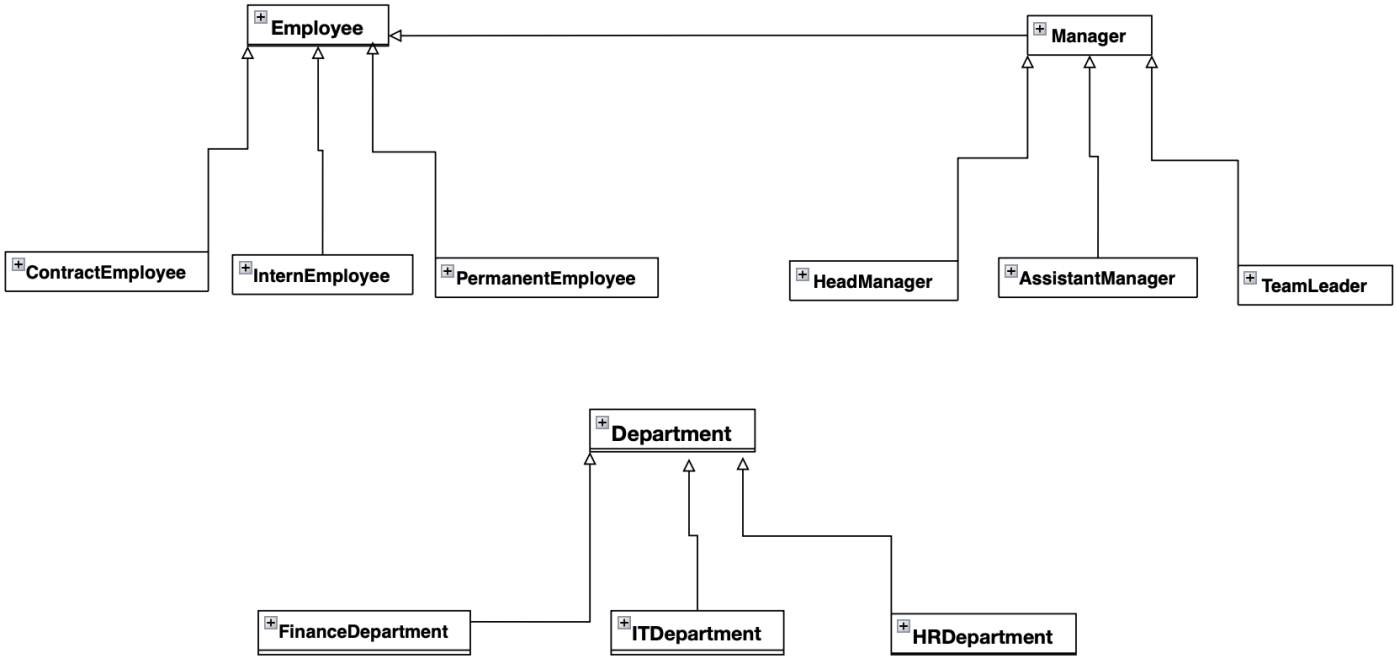
2. UML Model 1 – Class Diagram



I created the main structure for the Bank Organisation System before starting the full implementation in Java. I decided to separate the system into three main parent classes: Employee, Manager and Department, because these are the main parts of the organisation.

For each parent class, I also created three subclasses to represent different types inside the bank system and to demonstrate inheritance in Object-Oriented Programming. This helped me understand how child classes can inherit common attributes and methods from parent classes while still having their own specific information.

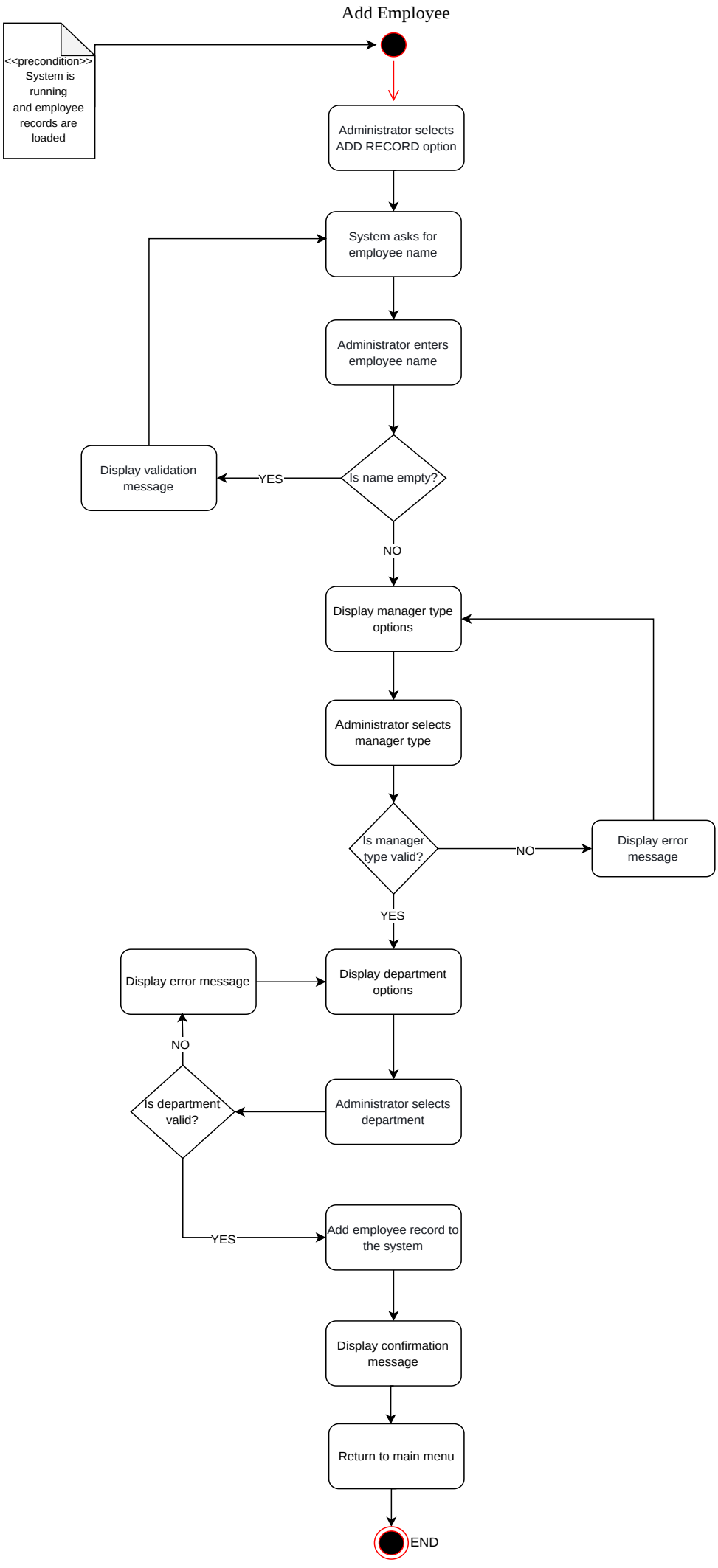
Creating the diagram before coding also helped me organise the project structure more clearly.



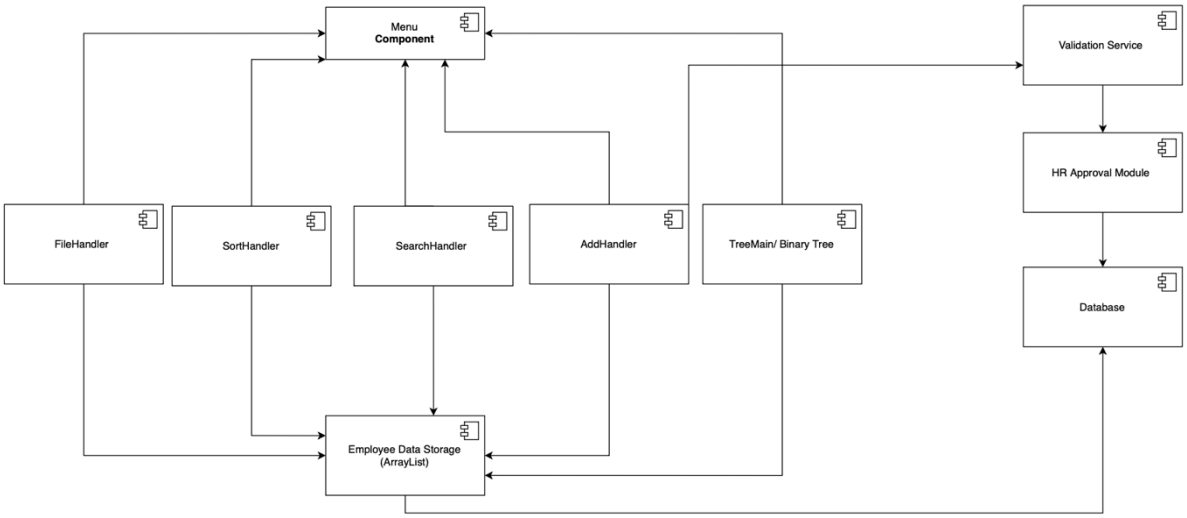
3. UML Model 2 - Activity Diagram

For the second UML model, I decided to create an Activity Diagram based on the "Add Employee Record" use case. I chose this use case because it includes both the normal system flow and different validation scenarios, such as empty input fields and invalid manager type or department selections.

This activity diagram helped me represent the sequence of actions, decision points and error handling within the system. I also used Yes/No conditions in the decision nodes to make the process easier to follow and more aligned with the examples during class.


Add Employee
<<precondition>>
System is running and employee records are loaded
Administrator selects ADD RECORD option
System asks for employee name
Administrator enters employee name
Is name empty?
YES
Display validation message
NO
Display manager type options
Administrator selects manager type
Is manager type valid?
NO
Display error message
YES
Display department options
Display error message
NO
Is department valid?
Administrator selects department
YES
Add employee record to the system
Display confirmation message
Return to main menu
END

4.UML Model 3 – Component Diagram



I created this Component Diagram to represent the main parts of the Bank Organisation System and how they interact with each other. I also included possible future components, such as validation, HR approval and database storage, to demonstrate how the system could be expanded in a larger banking environment.

5. Requirements

5.1 Functional Requirements

	Requirement	Justification
FR1	The system shall read and load employee records from the file Applicants_Form.txt	This is needed because the program needs initial employee data before it can sort, searching or building the hierarchy.
FR2	The system shall sort employee records alphabetically using a recursive sorting algorithm and display the first 20 results	This makes the list easier to read and prepares the list for efficient searching operations.
FR3	The system shall allow the user to search for an employee by name and return details such as manager type and department name	To help the user to quickly find employee information without manually checking the full list.
FR4	The system shall allow the user to add a new employee record, including name, manager type and department, and shall validate that the selected manager type and department already exist in the system	To keeps the employee list updated and helps prevent invalid data from being saved.

FR5	The system shall create and display an employee hierarchy using a binary tree structure, storing a minimum of 20 employee records and show the tree height and total node count	To helps the organisation structure and and also meets the binary tree requirement from the assignment.
FR6	The system shall provide a structured console menu using Enums, with options including SORT, SEARCH, ADD RECORD, CREATE BINARY TREE and EXIT.	This keeps the menu organised and easier to manage in the Java program.
FR7	The system shall display clear confirmation messages after completing actions such as reading, sorting, searching, adding records and creating the binary tree.	To help the administrator understand whether each action was completed successfully.

5.2 Non-Functional Requirements

	Requirements	Justification
NFR1	The system shall provide a clear and easy-to-use console menu, allowing the user to navigate between options without confusion.	This is important because the user needs to understand the available options and use the program without getting lost.
NFR2	The system shall display clear validation messages for invalid user input and prompt the user to try again.	This helps the user understand what went wrong and how to correct the input.
NFR3	The system shall maintain organised and readable code, with descriptive variable names and comments throughout, to ensure clarity and easier maintenance.	This makes the code easier to understand and maintain.
NFR4	The system shall be developed using Object-Oriented Programming principles, such as inheritance and encapsulation.	This helps keep the program structure organised and reduces repeated code.
NFR5	The system shall handle unexpected or invalid user input without crashing, ensuring the program continues to run normally.	This makes the system more reliable because it can continue running even when the user enters something wrong.
NFR6	The system shall respond to user actions faster without delay.	This improves the user experience because the program should feel quick and responsive when using the menu options.

6. User stories

User Story 1 – Read Employee Records

As a bank administrator, I want to load employee records from a file, so that I can access employee information such as name, department and job title within the system.

Acceptance Criteria

- The system asks for a filename.
- The administrator can enter a filename.
- The system reads the employee records successfully.
- An error message is displayed if the file does not exist.

User Story 2 – Sort Employee Records

As a bank administrator, I want to sort employee records alphabetically, so that employee information is easier to read and search.

Acceptance Criteria

- The administrator can select the SORT option.
- The system sorts employee records alphabetically.
- The first 20 sorted records are displayed.
- An error message is displayed if no records are loaded.

User Story 3 – Search Employee by Name

As a bank administrator, I want to search for an employee by name, so that I can quickly find their information such as department and manager type.

Acceptance Criteria:

- The administrator can enter an employee name.
- The system searches the employee records.
- The system displays the employee's department and manager type when found.
- A "not found" message is displayed if the employee does not exist.
- A validation message is displayed if the name field is empty.

User Story 4 – Add Employee Record

As a bank administrator, I want to add a new employee record, so that the employee database can be updated.

Acceptance Criteria

- The administrator can enter employee information.
- The system validates the entered data.
- The employee record is added successfully.
- Error messages are displayed for invalid input.

User Story 5 – Create Employee Hierarchy

As a bank administrator, I want to create an employee hierarchy using a binary tree structure, so that I can visualize the organisation structure more clearly.

Acceptance Criteria

- The administrator can select the CREATE TREE option.
- The system requires a minimum of 20 employee records to create the tree.
- The system displays the hierarchy using level-order traversal.
- The system displays the tree height and total node count.
- An error message is displayed if there are less than 20 employee records.

7. Unit Test

Read Employee Records

TEST ID NUMBER:	001	**Developer:**	Valeria Dionisio	**Date Test Carried Out:**	19/05/ 2026	**Test Name:**	Read Employee Records
Module Tested	*FileHandler / READ FILE menu option*						
Description of Test	*Check if the system can read the employee data file and store the records in memory*			**Test Carried out by:**	*Valeria Dionisio*		
Test Precondition(s)	File Applicants_Form – Sample data file for read.txt must exist in the project folder						
Dependencies (if any)	none						

TEST STEP	**DATA (Input)**	**Expected Result(s)**	**Actual Result(s)**	**PASS /FAIL**	**NOTES**
1.Run the program	Click Run Project	The menu is displayed	The menu was displayed	PASS	The program started correctly
2.Select option 1 – READ FILE	Option 1	The system opens and reads the employee file	The file was opened successfully.	PASS	The file path and filename were correct.
3.Check the terminal output	Terminal Output	Employee records are stored in memory using the ArrayList.	46 employee objects were stored.	PASS	This confirms the records are ready for sorting, searching and hierarchy creation.
4.Test invalid input	Incorrect file name/missing file	The system displays a file not found error and does not crash	Displayed “File not found”	PASS	This checks the exception path for file reading

Sort Employee Records

TEST ID NUMBER:	002	**Developer:**	Valeria Dionisio	**Date Test Carried Out:**	19/05/ 2026	**Test Name:**	Sort Employee Records
Module Tested	*SortHandler / SORT menu option*						
Description of Test	Check if the system can sort employee records alphabetically and display the first 20 sorted records.			**Test Carried out by:**	*Valeria Dionisio*		
Test Precondition(s)	Employee records must already be loaded into the system. (option read file)						
Dependencies (if any)	Read Employee Records test must pass first.						

TEST STEP	DATA (Input)	Expected Result(s)	Actual Result(s)	PASS /FAIL	NOTES
1.Run the program and load records	Option 1	Employee records are loaded into memory.	46 employee objects were stored.	PASS	Sorting depends on records being loaded first.
2.Select SORT option	Option 2	The system sorts employee records alphabetically.	SORT selected and records were sorted.	PASS	Recursive bubble sort was executed.
3.Check sorted output	Terminal Output	The first 20 sorted employee records are displayed.	The first 20 employees were displayed in alphabetical order.	PASS	This confirms the sorting output matches the brief.
4.Test sorting without loaded records	Option 2 before READ FILE	The system should not crash and should handle the empty list.	The program did not crash.	PASS	checks how the system behaves if no records are available.

Add Employee Validation

TEST ID NUMBER:	003	Developer:	Valeria Dionisio	Date Test Carried Out:	19/05/ 2026	Test Name:	Add Employee Validation
Module Tested	*AddHandler / ADD RECORD menu option*						
Description of Test	Check if the system validates employee input correctly before adding a new record.			Test Carried out by:	*Valeria Dionisio*		
Test Precondition(s)	The employee list must already be loaded into memory.						
Dependencies (if any)	Read Employee Records test must pass first.						

TEST STEP	DATA (Input)	Expected Result(s)	Actual Result(s)	PASS /FAIL	NOTES
1.Select ADD RECORD option	Option 4	The add employee menu is displayed.	The add employee menu was displayed successfully.	PASS	The system entered add mode correctly.
2. Add valid employee data	Valid name, email, manager type and department	The employee is added successfully to the ArrayList.	The employee was added successfully.	PASS	This confirms valid records can be stored
3. Test invalid first name	Name containing numbers	The system rejects the invalid name input.	Invalid name message displayed.	PASS	Validation prevents incorrect

					characters in names.
4. Test invalid email	Email without "@"	The system rejects the invalid email.	Invalid email message displayed.	PASS	Email validation was handled correctly.
5. Test invalid manager type	Invalid manager type	The system rejects the invalid manager type.	Invalid manager type message displayed.	PASS	This protects data consistency.
6. Test invalid department	Department that does not exist	The system rejects the invalid department.	Invalid department message displayed.	PASS	Department validation worked correctly.

Search Employee Records

TEST ID NUMBER:	004	**Developer:**	Valeria Dionisio	**Date Test Carried Out:**	19/05/ 2026	**Test Name:**	Search Employee Records
Module Tested	*SearchHandler / SEARCH menu option*						
Description of Test	Check if the system can search for an employee by name and return the correct employee information.			**Test Carried out by:**	*Valeria Dionisio*		
Test Precondition(s)	Employee records must already be loaded and sorted.						
Dependencies (if any)	Read Employee Records and Sort Employee Records tests must pass first.						

TEST STEP	**DATA (Input)**	**Expected Result(s)**	**Actual Result(s)**	**PASS /FAIL**	**NOTES**
1. Load employee records	Option 1	Employee records are loaded into memory.	46 employee objects were stored.	PASS	Search depends on loaded records.
2. Sort employee records	Option 2	Employee records are sorted alphabetically.	The first 20 sorted employees were displayed.	PASS	Binary search requires sorted data.
3. Search existing employee	Alex Turner	The system finds and displays the employee details.	Employee details were displayed successfully.	PASS	This confirms recursive binary search works.
4. Search non-existing employee	Fake Name	The system displays "Employee not found."	"Employee not found." was displayed.	PASS	This checks the exception path.

8. References

CCT College Dublin (2026) *Bubble_Sort_Explanation*. Moodle lecture material.

CCT College Dublin (2026) *EmployeeTree*. Moodle lecture material.

CCT College Dublin (2026) *MergeSort_Full_Explanation_and_Trace*. Moodle lecture material.

CCT College Dublin (2026) *Linear and Binary Search*. Moodle lecture material.

CCT College Dublin (2026) *UML Diagram Templates and Examples*. Moodle lecture material.

Lucidchart *UML Diagram Examples and Templates*. Available at: https://www.lucidchart.com/pages/uml-diagram-tool (Accessed: 05 May 2026).

Draw.io *UML Diagram Tool*. Available at: https://app.diagrams.net/ (Accessed: 02 May 2026).

Visual Paradigm *UML Diagram Examples*. Available at: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/ (Accessed: 02 May 2026).

Stack Overflow *Java Programming Discussions*. Available at: https://stackoverflow.com/questions/tagged/java (Accessed: 05 May 2026).

9. Appendix

I used ChatGPT mainly to help improve the grammar of my writing and organise my ideas throughout the report. I also used it to help me better understand some UML concepts and how to use Draw.io.

There were also moments where I struggled to understand how to use Draw.io correctly, especially when trying to identify the correct UML shapes, symbols and connections. Because of this, I used ChatGPT for guidance on where to find certain Draw.io functionalities and to better understand which diagram components were appropriate for each UML model.

The templates used for the use cases, unit tests and UML models were originally provided by the lecturer through Moodle materials and classroom examples. Several examples and explanations were also demonstrated during class sessions, and additional questions about improving the diagrams and system modelling were clarified in person throughout the development process.

The support was mainly focused on clarification and guidance, while the final diagrams, system decisions, testing and implementation were completed and adapted by me.